

SESSION 6

RTK Query

คิวรี่ · มิวเทชัน · แท็กแคช · สถานะการโหลดอัตโนมัติ

หลักสูตร React-Redux · โปรเจกต์ AcadeMate

Dr. Watcharin Sarachai

createApi

บริการ API เดียว ครอบคลุมทุก Endpoint

useQuery hooks

แทนที่ useEffect + fetch

useMutation hooks

สถานะโหลด/ข้อผิดพลาดในตัว

Cache & Tags

โหลดซ้ำอัตโนมัติเมื่อแคชถูก Invalidate

Session 6 — เนื้อหาและเป้าหมายการเรียนรู้

ส่วนที่ 1 — ทำไมต้อง RTK Query?

เข้าใจต้นทุนของ Thunk แบบ Manual และสิ่งที่ RTK Query จัดการให้อัตโนมัติ

ส่วนที่ 2 — createApi

เขียน API Slice แรกพร้อม Endpoint สำหรับ Query และ Mutation และกำหนดค่า Store

ส่วนที่ 3 — Queries & Mutations

ใช้ Hook useQuery และ useMutation พร้อมสถานะโหลด ข้อผิดพลาด และสำเร็จอัตโนมัติ

ส่วนที่ 4 — Cache Tags

โหลดข้อมูลล่าสุดด้วย providesTags และ invalidatesTags

Lab 6 — AcadeMate RTK Query

แทนที่ Thunk ทั้งหมดด้วยบริการ studentsApi เดียว ไม่ต้องเขียน Fetch Boilerplate อีกต่อไป

ทำไมต้อง RTK Query?

ทุก Pattern ของ useEffect + fetch + dispatch มี Boilerplate เหมือนกัน RTK Query จัดส่วนนี้

ต้นทุนของการดึงข้อมูลแบบ Manual

ทุก Entity (นักศึกษา วิชา เกรด) ต้องการ Boilerplate เหมือนกัน — ซ้ำกันสามครั้งใน AcadeMate:



Pattern Manual Thunk (Session 4)

```
// 1. studentsThunks.js – createAsyncThunk
export const fetchStudents = createAsyncThunk(
  'students/fetchAll',
  async (_, { rejectWithValue }) => {
    try {
      const res = await fetch(API_URL);
      return await res.json();
    } catch (e) { return rejectWithValue(e.message); }
  });

// 2. Extra fields in studentsSlice.js
status: 'idle', error: null

// 3. Three extraReducers cases
(pending/fulfilled/rejected)

// 4. useEffect in every component
useEffect(() => {
  if (status === 'idle') dispatch(fetchStudents());
}, [status, dispatch]);

// ← Repeat for addStudentAsync, updateStudentAsync,
//   deleteStudentAsync ... x4 thunks per entity
```



RTK Query (Session 6)

```
// 1. One endpoint in studentsApi.js
getStudents: builder.query({
  query: () => 'students',
  providesTags: ['Student'],
}),

// 2. No extra state needed
//   RTK Query owns loading + error + data

// 3. No extraReducers at all

// 4. One hook in the component
const {
  data: students = [],
  isLoading,
  isError,
} = useGetStudentsQuery();

// ← Add/update/delete: one mutation each
//   Cache auto-invalidates & re-fetches
```

สิ่งที่ RTK Query จัดการให้อัตโนมัติ

RTK Query คือชั้นการดึงข้อมูลและแคชที่ผสานอยู่ใน Redux Toolkit — ไม่ต้องติดตั้งแพ็คเกจเพิ่มเติม:

สถานะโหลดอัตโนมัติ

isLoading, isFetching, isSuccess, isError — ต่อ Query ต่อ Mutation ไม่ต้องกำหนด Status Field เอง

การหลีกเลี่ยง Request ซ้ำ

สับ Component ที่เรียก Query เดียวกันใช้ Request เดียวร่วมกัน ไม่ใช่สับ Network Call แยกกัน

Hook ที่สร้างอัตโนมัติ

useGetStudentsQuery(), useAddStudentMutation() — ตั้งชื่อตาม Endpoint Definition อัตโนมัติ

แคชโดยค่าเริ่มต้น

ผลลัพธ์ถูกแคชใน Redux โหลดซ้ำเมื่อแคชหมดอายุหรือแท็กถูก Invalidate เท่านั้น

การโหลดซ้ำอัตโนมัติ

Invalidate แท็ก — ทุก Query ที่ Provide แท็กนั้นโหลดซ้ำอัตโนมัติ ไม่ต้อง Dispatch เอง

ไม่ต้องใช้ useEffect เพื่อดึงข้อมูล

ไม่ต้อง Dispatch ใน useEffect Hook ผูก Component กับข้อมูลในแคชโดยตรง

SESSION 6

createApi

การเรียกฟังก์ชันครั้งเดียวกำหนด API Service ทั้งหมด — Base URL, Endpoint และกฎแคช

createApi — โครงสร้างและส่วนประกอบ

createApi() สร้าง Redux Slice, Hook อัตโนมัติ และการเชื่อมต่อ Middleware — ทั้งหมดจาก Config Object เดียว:

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";

export const studentsApi = createApi({
  // ① Unique key — where the cache lives in the Redux store
  reducerPath: "studentsApi",

  // ② How to make HTTP requests (baseUrl applied to all endpoints)
  baseQuery: fetchBaseQuery({ baseUrl: "https://api.mockapi.io/v1/" }),

  // ③ Entity tag types used for cache invalidation
  tagTypes: ["Student"],

  // ④ All endpoints — queries (reads) and mutations (writes)
  endpoints: (builder) => ({
    // builder.query() → cached read, generates useXxxQuery()
    // builder.mutation() → write, generates useXxxMutation()
  }),
});

// ⑤ Auto-exported hooks (consumed in components)
export const { useGetStudentsQuery, useAddStudentMutation } = studentsApi;
```

fetchBaseQuery — Base URL และตัวเลือก

fetchBaseQuery ห่อหุ้ม Fetch API ของเบราว์เซอร์ด้วย baseUrl, Header เริ่มต้น และการจัดการข้อผิดพลาด:

การตั้งค่าพื้นฐาน (AcadeMate Lab 6)

```
const baseQuery = fetchBaseQuery({
  baseUrl: 'https://api.mockapi.io/v1/',
});
// Endpoint query strings append to baseUrl:
// 'students'      → .../v1/students
// `students/${id}` → .../v1/students/42
```

พร้อม Auth Header (JWT)

```
const baseQuery = fetchBaseQuery({
  baseUrl: '/api/',
  prepareHeaders: (headers, { getState }) => {
    const token = getState().auth.token;
    if (token)
      headers.set('Authorization', `Bearer ${token}`);
    return headers;
  },
});
```

ลอจิก Retry แบบกำหนดเอง

```
import { retry } from '@reduxjs/toolkit/query/react';

// Retry failed requests up to 3 times
const baseQueryWithRetry = retry(
  fetchBaseQuery({ baseUrl: '/api/' }),
  { maxRetries: 3 }
);

export const api = createApi({
  baseQuery: baseQueryWithRetry,
  // ...
});
```



สำหรับ AcadeMate Lab 6

ใช้ baseUrl ของ mockapi.io จาก Session 4 เดิม ไม่ต้องใช้ Auth Header เน้นที่ Pattern Query/Mutation/Tag — ให้การตั้งค่าเรียบง่าย

Endpoint — builder.query() เทียบกับ builder.mutation()

ใช้ query() สำหรับการอ่าน (GET) และ mutation() สำหรับการเขียน (POST/PUT/DELETE):

builder.query() — การอ่านแบบมีแคช

```
getStudents: builder.query({
  // query() returns the URL string
  // or a RequestConfig object
  query: () => 'students',
  // Tags this cached result provides:
  providesTags: ['Student'],
}),

// With an argument (cached per id):
getStudentById: builder.query({
  query: id => `students/${id}`,
  providesTags: (result, err, id) =>
    [{ type: 'Student', id }],
}),
```

builder.mutation() — การเขียน

```
addStudent: builder.mutation({
  // Returns a RequestConfig object:
  query: student => ({
    url: 'students', method: 'POST',
    body: student,
  }),
  // Invalidate the 'Student' cache:
  invalidatesTags: ['Student'],
}),

deleteStudent: builder.mutation({
  query: id => ({
    url: `students/${id}`,
    method: 'DELETE',
  }),
  invalidatesTags: ['Student'],
}),
```



ฟังก์ชัน Query รับอาร์กิวเมนต์ได้เพียง 1 ตัว ส่ง Object สำหรับหลายค่า: query: ({id, body}) => ({ url: `students/\${id}`, method: 'PUT', body })

Hook ที่สร้างอัตโนมัติ — รูปแบบการตั้งชื่อ

RTK Query สร้าง Hook จากชื่อ Endpoint ตาม Pattern นี้เราจะรู้ชื่อ Hook เสมอ:

ชื่อ Endpoint	ประเภท	Hook ที่สร้าง	วิธีเรียกใช้
getStudents	query	<code>useGetStudentsQuery()</code>	ไม่มีอาร์กิวเมนต์ — Subscribe ตอน Mount
getStudentById	query	<code>useGetStudentByIdQuery(id)</code>	ส่ง id — แคชแยกต่อ id
addStudent	mutation	<code>useAddStudentMutation()</code>	<code>const [add, result] = hook()</code>
updateStudent	mutation	<code>useUpdateStudentMutation()</code>	<code>trigger({id, ...patch})</code>
deleteStudent	mutation	<code>useDeleteStudentMutation()</code>	<code>trigger(id)</code>

สูตร: `use` + [ชื่อ Endpoint ตัวพิมพ์ใหญ่] + `Query` หรือ `Mutation`

ตัวอย่าง: Endpoint 'addStudent' → `useAddStudentMutation()` | Endpoint 'getStudents' → `useGetStudentsQuery()`

Queries

Hook useQuery ผูก Component กับข้อมูลในแคชและจัดการสถานะโหลด/ข้อผิดพลาดอัตโนมัติ

useQuery — ค่าที่ Hook คืนกลับ

ทุก Hook useQuery คืน Object พร้อม Flag สถานะ, ข้อมูลในแคช และฟังก์ชัน Refetch:

ค่าที่คืนกลับสำคัญ

```
const {
  data,           // The cached result (undefined until loaded)
  isLoading,     // true on first load (no cache yet)
  isFetching,    // true on ANY request incl. background
  isSuccess,     // true when data is available
  isError,       // true when the request failed
  error,         // the error object
  refetch,       // call to force a manual re-fetch
} = useGetStudentsQuery();
```

```
// isLoading vs isFetching:
// isLoading = first load only (spinner)
// isFetching = every network request
//           (use for polling indicators)
```

Pattern การใช้งานใน Component

```
function StudentTable() {
  const {
    data: students = [],
    isLoading,
    isError,
  } = useGetStudentsQuery();

  if (isLoading) return <p>Loading...</p>;
  if (isError)   return <p>Failed to load.</p>;

  return (
    <table>
      <tbody>
        {students.map(s => (
          <tr key={s.id}>
            <td>{s.name}</td>
            <td>{s.gpa}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Query แบบมีพารามิเตอร์ — การเลือกตามอาร์กิวเมนต์

ส่งอาร์กิวเมนต์ให้ Hook — RTK Query แคมป์แยกกันต่อค่าอาร์กิวเมนต์ที่ไม่ซ้ำกัน:

นิยาม Endpoint

```
// In studentsApi.js
getStudentById: builder.query({
  query: id => `students/${id}`,
  // Separate cache entry per id:
  providesTags: (result, err, id) =>
    [{ type: 'Student', id }],
}),

// Filter by minimum GPA:
getHighAchievers: builder.query({
  query: minGpa =>
    `students?gpa_gte=${minGpa}`,
  providesTags: ['Student'],
}),
```

การใช้งานใน Component

```
// StudentRow.jsx — O(1) lookup by id
function StudentRow({ id }) {
  const { data: student, isLoading } = useGetStudentByIdQuery(id);

  if (isLoading) return <tr><td>...</td></tr>;

  return (
    <tr>
      <td>{student.name}</td>
      <td>{student.gpa}</td>
    </tr>
  );
}

// Skip pattern — don't fetch until id exists:
useGetStudentByIdQuery(id, { skip: !id })

// HighAchieversList.jsx
function HighAchieversList() {
  const { data = [] } = useGetHighAchieversQuery(3.5);
  return <ul>{data.map(s =>
    <li key={s.id}>{s.name}</li>)}</ul>;
}
```

Mutations

Hook useMutation คือนฟังก์ชัน Trigger — เรียกใช้เมื่อผู้ใช้ดำเนินการ

useMutation — Signature และ Pattern ของ Hook

useMutation คืน Tuple: [triggerFn, responseObject] เรียก triggerFn พร้อมข้อมูล Payload:

Signature ของ Hook และ unwrap

```
const [
  addStudent,    // trigger function
  {
    isLoading,   // true while in-flight
    isSuccess,   // true after success
    isError,     // true after failure
    error,       // error object
    data,        // server response body
    reset,       // clears result state
  }
] = useAddStudentMutation();

// Fire the request:
addStudent({ name: 'Alice', gpa: 3.8 });

// With unwrap() — throws on error:
try {
  const result = await addStudent({ name, gpa }).unwrap();
  // navigate or reset form
} catch (err) {
  console.error(err);
}
```

Component AddStudentForm

```
function AddStudentForm() {
  const [name, setName] = useState('');
  const [gpa, setGpa] = useState('');
  const [addStudent, { isLoading }] = useAddStudentMutation();

  const onSubmit = async e => {
    e.preventDefault();

    await addStudent({
      name, gpa: Number(gpa)
    }).unwrap();

    setName(''); setGpa('');
  };

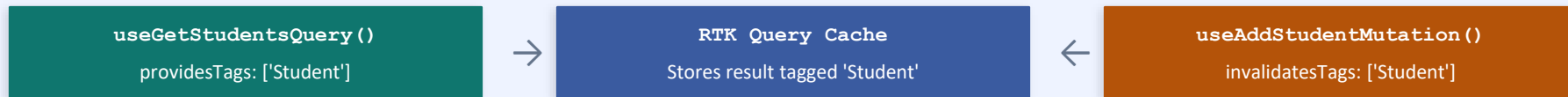
  return (
    <form onSubmit={onSubmit}>
      <input value={name}
        onChange={e => setName(e.target.value)}
        placeholder="Name" />
      <input value={gpa}
        onChange={e => setGpa(e.target.value)}
        placeholder="GPA" />
      <button disabled={isLoading}>
        {isLoading ? 'Saving...' : 'Add Student'}
      </button>
    </form>
  );
}
```

Cache Tags

แท็กเชื่อมโยง Mutation กับ Query — Mutation Invalidate แท็ก ทุก Query ที่ Provide แท็กนั้นโหลดซ้ำ

Cache Tags — กลไกการ Invalidation

แท็กคือป้ายกำกับที่เชื่อมผลลัพธ์ Query (providesTags) กับผลข้างเคียงของ Mutation (invalidatesTags):



1 Component Mount

`useGetStudentsQuery()` ทำงาน → ดึงข้อมูล → แคชถูกติดแท็ก 'Student'

2 ผู้ใช้เพิ่มนักศึกษา

Component เรียก `addStudent(data).unwrap()`

3 Mutation สำเร็จ

RTK Query พบ `invalidatesTags: ['Student']` → ทำเครื่องหมายแคชว่าล้าสมัย

4 โหลดซ้ำอัตโนมัติ

`getStudents` ให้แท็ก 'Student' → เรียก GET ใหม่อัตโนมัติ

5 UI อัปเดต

`StudentTable` Render ใหม่พร้อมข้อมูลล่าสุด — ไม่ต้อง Dispatch เอง

providesTags & invalidatesTags — Patterns

ใช้ Pattern แบบง่ายสำหรับแอปทั่วไป Pattern แบบละเอียดหลีกเลี่ยงการโหลดรายการทั้งหมดซ้ำตอนอัปเดต:

Pattern แบบง่าย (Invalidate ทั้งหมด)

```
getStudents: builder.query({
  query: () => 'students',
  providesTags: ['Student'],
}),

addStudent: builder.mutation({
  query: s => ({
    url: 'students', method: 'POST', body: s }),
  // Invalidate ALL 'Student' tagged queries:
  invalidatesTags: ['Student'],
}),

deleteStudent: builder.mutation({
  query: id => ({
    url: `students/${id}`, method: 'DELETE' }),
  invalidatesTags: ['Student'],
}),
```

ใช้ Pattern ใดใน Lab 6?

ใช้แบบใดก็ได้ แต่ Pattern แบบละเอียดมักใช้การแท็กละเอียด ซึ่งเป็นการ Optimize สำหรับรายการขนาดใหญ่ (100+ รายการ) เพื่อหลีกเลี่ยงการโหลด Collection ทั้งหมดซ้ำทุกครั้งที่อัปเดต

Pattern แบบละเอียด (List + Item)

```
getStudents: builder.query({
  query: () => 'students',
  providesTags: result =>
    result
    ? [...result.map(({ id }) => ({ type: 'Student', id })),
      { type: 'Student', id: 'LIST' }]
    : [{ type: 'Student', id: 'LIST' }],
}),

updateStudent: builder.mutation({
  query: ({ id, ...patch }) => ({
    url: `students/${id}`,
    method: 'PUT', body: patch }),
  // Only re-fetch the changed student:
  invalidatesTags: ({ id }) =>
    [{ type: 'Student', id }],
}),
```

SESSION 6

การตั้งค่า Store

เพิ่ม Reducer และ Middleware ของ API ลงใน configureStore — สอนบรรทัดและการเชื่อมต่อเสร็จสมบูรณ์

configureStore — การเพิ่ม RTK Query

RTK Query ต้องลงทะเบียน Reducer และ Middleware ที่สร้างขึ้น — หากไม่มี Middleware แท้ก็จะไม่ทำงาน:

src/app/store.js

```
export default AddStudentForm;
import { configureStore } from '@reduxjs/toolkit';
import { studentsApi }
  from '../features/students/studentsApi';
export const store = configureStore({
  reducer: {
    // ① Add the API slice reducer
    // (manages cache, request states, tags)
    [studentsApi.reducerPath]: studentsApi.reducer,
  },
  middleware: getDefault =>
    // ② Append RTK Query middleware
    // NEVER replace — always .concat()
    getDefault().concat(studentsApi.middleware),
});
```

reducerPath

studentsApi.reducerPath คือสตริง 'studentsApi' การใช้เป็น Computed Key ทำให้ Store และนิยาม API สอดคล้องกัน

studentsApi.reducer

Reducer ที่สร้างอัตโนมัติจัดการแคช: สถานะ Request, Payload ข้อมูล, แท็ก Entity และจำนวน Subscription

studentsApi.middleware

ต้องเพิ่มเข้าไป เปิดใช้อายุแคช, Polling และการ Invalidate แท็ก หากไม่มี invalidatesTags จะไม่มีผล

ใช้ concat() ไม่ใช่ replace

getDefault() รักษา redux-thunk และ Redux DevTools ไว้ ใช้ concat() เสมอ — ห้าม Overwrite

SESSION 6

Lab 6 — AcadeMate RTK Query

แทนที่ Thunk แบบ Manual ทั้งหมดด้วยบริการ studentsApi เดียว ไม่ต้องมี Loading State ใน Slice อีกต่อไป

Lab 6 — ภาพรวมและการเปลี่ยนแปลงไฟล์

การเปลี่ยนแปลงสิ่งจุด — UI และ Component ไม่เปลี่ยน เฉพาะชั้นข้อมูลถูกแทนที่:

สร้างใหม่ `src/features/students/studentsApi.js`

ไฟล์ใหม่ — createApi พร้อม Endpoint: `getStudents`, `getStudentById`, `addStudent`, `updateStudent`, `deleteStudent`

แก้ไข `src/app/store.js`

เพิ่ม `studentsApi.reducer` และ `studentsApi.middleware` ลบ `studentsSlice` ออกทั้งหมด — ไม่ต้องจัดการ Loading State เองอีกต่อไป

แก้ไข `src/components/StudentTable.jsx`

แทนที่ `useSelector` + `useEffect` + `dispatch` ด้วย `useGetStudentsQuery()` ลบการแสดง Status/Error ออก

แก้ไข `src/components/AddStudentForm.jsx`

แทนที่ `dispatch(addStudentAsync())` ด้วย `useAddStudentMutation()` การ Invalidate แคชทำงานอัตโนมัติ

Lab 6 — ขั้นตอนที่ 1: สร้าง studentsApi (ส่วนที่ 1/2)

ขั้นตอนที่ 1: สร้าง src/features/students/studentsApi.js พร้อม Config พื้นฐานและ Query Endpoint

```
// src/features/students/studentsApi.js
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

const BASE = 'https://api.mockapi.io/v1/';

export const studentsApi = createApi({
  reducerPath: 'studentsApi',
  baseQuery: fetchBaseQuery({ baseUrl: BASE }),
  tagTypes: ['Student'],
  endpoints: builder => ({
    // — QUERIES (reads, results are cached) —————
    getStudents: builder.query({
      query: () => 'students',
      providesTags: (result) =>
        result
          ? [...result.map(({ id }) => ({ type: "Student", id })), { type: "Student", id: "LIST" }]
          : [{ type: "Student", id: "LIST" }],
    }),
    getStudentById: builder.query({
      query: (id) => `students/${id}`,
      providesTags: (result, error, id) => [{ type: "Student", id }],
    }),
  })
```

Lab 6 — ขั้นตอนที่ 1: studentsApi (ส่วนที่ 2/2)

เพิ่ม Mutation Endpoint สามตัว จากนั้น Export Hook ที่สร้างอัตโนมัติด้านล่าง

```
// — MUTATIONS (writes, place inside endpoints block) —
addStudent: builder.mutation({ query: (student) => ({ url: "students", method: "POST", body: student }),
  invalidatesTags: [{ type: "Student", id: "LIST" }],
}),
updateStudent: builder.mutation({ query: (student) => ({ url: `students/${student.id}`, method: "PUT", body: student }),
  invalidatesTags: (result, error, student) => [{ type: "Student", id: student.id }, { type: "Student", id: "LIST" }],
}),
deleteStudent: builder.mutation({ query: (id) => ({ url: `students/${id}`, method: "DELETE" }),
  invalidatesTags: (result, error, id) => [{ type: "Student", id }, { type: "Student", id: "LIST" }],
}),
}), // ← closes endpoints: builder => ({
}); // ← closes createApi({

export const {
  useGetStudentsQuery, useGetStudentByIdQuery,
  useAddStudentMutation, useUpdateStudentMutation,
  useDeleteStudentMutation,
} = studentApi;
```

Lab 6 — ขั้นตอนที่ 2: Selector

ขั้นตอนที่ 2: ปรับ selectors.js

selectors.js — ใช้ StudentAPI

```
import { createSelector } from "@reduxjs/toolkit";
import { studentApi } from "../studentApi";

const selectStudentsResult = studentApi.endpoints.getStudents.select();
const selectStudentsData = createSelector(selectStudentsResult, (result) => result.data ?? []);
export const selectAverageGpa = createSelector(selectStudentsData, (students) => {
  if (students.length === 0) return "-";
  return (students.reduce((acc, s) => acc + s.gpa, 0) / students.length).toFixed(2);
});
export const selectStudentCount = createSelector(selectStudentsData, (students) => students.length);
export const selectMaxGPA = createSelector(selectStudentsData, (students) => {
  if (students.length === 0) return 0;
  return Math.max(...students.map((s) => s.gpa)).toFixed(2);
});
export const selectMinGPA = createSelector(selectStudentsData, (students) => {
  if (students.length === 0) return 0;
  return Math.min(...students.map((s) => s.gpa)).toFixed(2);
});
```

Lab 6 — ขั้นตอนที่ 3: Store

ขั้นตอนที่ 3: เชื่อมต่อ studentsApi เข้า Store จากนั้นแทนที่การดึงข้อมูลแบบ Manual ของ StudentTable

store.js — เพิ่ม API Slice

```
import { configureStore } from '@reduxjs/toolkit';
import { studentsApi } from
  '../features/students/studentsApi';

export const store = configureStore({
  reducer: {
    [studentsApi.reducerPath]: studentsApi.reducer,
    courses: coursesReducer,
    grades: gradesReducer,
  },
  middleware: g =>
    g().concat(studentsApi.middleware),
});
```

ลบไฟล์ studentsSlice.js และ studentTrunk.js

ขั้นตอนที่ 4: StudentTable

StudentTable.jsx — แทนที่ useEffect

```
import { useState } from "react";
import {
  useDeleteStudentMutation,
  useGetStudentsQuery,
  useUpdateStudentMutation,
} from "../features/students/studentApi";
import EditModal from "../EditModal";
import StudentRow from "../StudentRow";

function StudentTable() {
  const { data: students = [], isLoading, isError,
    error, refetch } = useGetStudentsQuery();
  const [deleteStudent] = useDeleteStudentMutation();
  const [updateStudent] = useUpdateStudentMutation();
  const [editing, setEditing] = useState(null);
```

Lab 6 — ขั้นตอนที่ 4: StudentTable 2/3

ขั้นตอนที่ 4: StudentTable

StudentTable.jsx

```
const handleDelete = async (id) => {
  if (window.confirm("Are you sure you want to delete this student?")) {
    await deleteStudent(id);
  }
};

const handleEditSave = async (student) => {
  await updateStudent({ ...student, gpa: parseFloat(student.gpa) || 0 });
  setEditing(null);
};

if (isLoading) { return <div className="spinner">Loading...</div>; }
if (isError) {
  return (
    <div className="error-banner">
      <p>Error: {error?.status || "Failed to fetch students"}</p>
      <button onClick={refetch}>Retry</button>
    </div>
  );
}
```

ขั้นตอนที่ 4: StudentTable

StudentTable.jsx

```
if (!students || students.length === 0) { return <p>No students found. Add one to the list.</p>; }
return (
  <>
    <table className="student-table">
      <thead><tr><th>#</th><th>Name</th><th>Student ID</th><th>Major</th><th>GPA</th><th>Actions</th></tr></thead>
      <tbody>
        {students.map((student, index) => (
          <StudentRow key={student.id} student={student} index={index} setEditing={setEditing} handleDelete={handleDelete}/>
        ))}
      </tbody>
    </table>
    {editing && (<EditModal student={editing} onSave={handleEditSave} onCancel={() => setEditing(null)} />
  )}
  </>
);
}
export default StudentTable;
```

Lab 6 — ขั้นตอนที่ : StudentRow

ขั้นตอนที่ 5: StudentRow

StudentRow.jsx

```
function StudentRow({ student, index, setEditing, handleDelete }) {  
  if (!student) return null;  
  return (  
    <tr className={student.gpa >= 3.5 ? "high-gpa" : ""}>  
      <td>{index + 1}</td><td>{student.name}</td><td>{student.studentId}</td><td>{student.major}</td>  
      <td className="gpa-cell">{student.gpa.toFixed(2)}</td>  
      <td>  
        <button type="button" onClick={() => setEditing(student)}>Edit</button>  
        <button type="button" onClick={() => handleDelete(student.id)}>Delete</button>  
      </td>  
    </tr>  
  );  
}  
  
export default StudentRow;
```

ขั้นตอนที่ 6: GpaSummary

GpaSummary.jsx

```
import { useSelector } from "react-redux";
import {
  selectAverageGpa,
  selectHighAchievers,
} from "../features/students/selectors";
import { useGetStudentsQuery } from "../features/students/studentApi";

function GpaSummary() {
  const { data: students = [] } = useGetStudentsQuery();
  const count = students.length;
  const avgGpa = useSelector(selectAverageGpa);
  const highList = useSelector(selectHighAchievers);

  ...
}
```

Lab 6 — ขั้นตอนที่ 3: AddStudentForm พร้อม useMutation

ขั้นตอนที่ 7: แทนที่ dispatch(addStudentAsync()) ด้วย Mutation Hook การ Invalidate แดชทำงานอัตโนมัติ



Session 4 — การ Dispatch Thunk

```
import { useDispatch } from 'react-redux';
import { addStudentAsync }
  from '../features/students/studentsThunks';

function AddStudentForm() {
  const dispatch = useDispatch();

  const handleSubmit = async e => {
    e.preventDefault();

    await dispatch(
      addStudentAsync({ name, gpa })
    ).unwrap();

    setName(''); setGpa('');
    // Must also handle status/error from slice
    // dispatch(fetchStudents()) if list not updated
  };
  // ...
}
```



Session 6 — Mutation Hook

```
import { useAddStudentMutation } from
  '../features/students/studentsApi';

function AddStudentForm() {
  const [addStudent, { isLoading }] = useAddStudentMutation();

  const onSubmit = async e => {
    e.preventDefault();
    await addStudent({
      name, gpa: Number(gpa)
    }).unwrap();
    setName(''); setGpa('');
  };
  // ✓ No useDispatch, no useSelector
  // ✓ No manual loading/error state
  // ✓ invalidatesTags auto-refetches list
  return (
    <form onSubmit={onSubmit}>
      {/* inputs unchanged */}
      <button disabled={isLoading}>
        {isLoading ? 'Saving...' : 'Add Student'}
      </button>
    </form>
  );
}
```

Lab 6 — ขั้นตอนที่ : AddStudentForm 1/2

ขั้นตอนที่ 7: AddStudentForm

AddStudentForm.jsx

```
import { useState } from "react";
import { useAddStudentMutation } from
"../features/students/studentApi";

const EMPTY_FORM = { name: "", studentId: "", major: "", gpa:
"" };

function AddStudentForm() {
  const [addStudent] = useAddStudentMutation();
  const [formData, setFormData] = useState(EMPTY_FORM);
  const [errors, setErrors] = useState({});

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    // Validation logic here
    if (!formData.name.trim() || !formData.studentId.trim()
    || formData.gpa === "" ) {
```

```
      setErrors({
        name: !formData.name.trim() ? "Name is required" : "",
        studentId: !formData.studentId.trim()
          ? "Student ID is required" : "",
        gpa: formData.gpa === "" ? "GPA is required" : "",
      });
      return;
    }
    const gpaNum = parseFloat(formData.gpa);
    if (isNaN(gpaNum) || gpaNum < 0 || gpaNum > 4) {
      setErrors((prev) => ({...prev, gpa: "GPA must be a number
between 0.0 and 4.0" }));
      return;
    }
    addStudent({ ...formData, gpa: gpaNum });
    setFormData(EMPTY_FORM);
    setErrors({});
  }
  return (...
```

Lab 6 — ขั้นตอนที่ : AddStudentForm 1/2

ขั้นตอนที่ 8: App

App.jsx

```
import "../App.css";
import GpaSummary from "../components/GpaSummary.jsx";
import AddStudentForm from "../components/AddStudentForm.jsx";
import StudentTable from "../components/StudentTable.jsx";

function App() {
  return (
    <div className="app-container">
      <header className="app-header"><h1>AcadeMate</h1>
      <p>Student Academic Performance Tracker – Session 2 Redux Migration</p>
    </header>
    <main className="app-main">
      <GpaSummary /> <AddStudentForm /> <StudentTable />
    </main>
  </div>
);
}
export default App;
```

RTK Query เทียบกับ Manual Thunk — เปรียบเทียบ

สำหรับการดึงข้อมูล RTK Query ลดโค้ดได้ ~60% และจัดข้อบกพร่องหลายประเภท:

ประเด็น	Manual Thunk (Session 4)	RTK Query (Session 6)
สถานะโหลด	<code>status:'loading'</code> ใน <code>Slice</code> + <code>extraReducers</code>	<code>isLoading</code> จาก Hook — อัตโนมัติ
สถานะข้อผิดพลาด	<code>Field error</code> + <code>rejected addCase</code>	<code>isError</code> + <code>error</code> จาก Hook — อัตโนมัติ
แคชข้อมูล	<code>list[]</code> ใน <code>Slice</code> — โหลดซ้ำทุกครั้งที่ Mount	แคชใน Store — TTL เริ่มต้น 60 วินาที
โหลดข้อมูลตอน Mount	<code>useEffect</code> + <code>dispatch(fetchStudents())</code>	<code>useGetStudentsQuery()</code> — อัตโนมัติ
โหลดซ้ำหลังเขียนข้อมูล	<code>Dispatch fetchStudents()</code> ซ้ำด้วยตนเอง	<code>invalidatesTags</code> สั่งให้โหลดซ้ำ
การหลีกเลี่ยงซ้ำ	Mount หลายครั้ง = Request หลายครั้ง	ใช้ <code>Subscription</code> ร่วมกัน — Request เดียว
โค้ดต่อ Entity	~80 บรรทัด (Thunk + Slice + <code>useEffect</code>)	~25 บรรทัด (Endpoint + Export Hook)
DevTools	ดู Action ใน Redux DevTools	ดู Query + แคชใน Tab RTK Query

ทดสอบความเข้าใจ

คำถาม:

มี Query ที่ Active อยู่สองตัว — `useGetStudentsQuery()` และ `useGetHighAchieversQuery(3.5)` — ทั้งคู่มี `providesTags: ['Student']` เมื่อผู้ใช้เรียก `deleteStudent(id)` ซึ่งมี `invalidatesTags: ['Student']` จะเกิดอะไรขึ้น?

```
// A: Only useGetStudentsQuery re-fetches — it targets the full list.  
// B: Both queries re-fetch — both provide the 'Student' tag that was invalidated.  
// C: Neither re-fetches — you must call refetch() manually.  
// D: Only the deleted item's cache entry is cleared, everything else is untouched.
```

คำตอบ:

B — ทั้งสอง Query โหลดซ้ำ ทุก Query ที่ Provide แท็กที่ถูก Invalidate จะโหลดซ้ำ โดยไม่คำนึงว่า Item ใดถูกเปลี่ยน ใช้แท็ก `{ type: 'Student', id }` สำหรับการควบคุมละเอียด

ข้อสังเกตสำคัญ:

`invalidatesTags: ['Student']` Invalidate ทุก Query ที่มีแท็ก 'Student' — ไม่ใช่แค่ตัวที่เปลี่ยนแปลง

แนวคิดสำคัญที่ครอบคลุมใน Session 6:

createApi

กำหนด API Service: reducerPath, baseQuery, tagTypes และ Endpoint ทั้งหมดในที่เดียว

fetchBaseQuery

ห่อหุ้ม fetch ด้วย baseUrl เพิ่ม prepareHeaders สำหรับ Auth และ retry() เพื่อความทนทาน

builder.query()

Endpoint GET แบบมีแคช ใช้ providesTags ดัดแปลงผลลัพธ์ สร้าง Hook useXxxQuery() อัตโนมัติ

builder.mutation()

Endpoint สำหรับเขียน (POST/PUT/DELETE) ใช้ invalidatesTags สร้าง Hook useXxxMutation() อัตโนมัติ

Cache Tags

providesTags ดัดแปลงข้อมูล Query invalidatesTags บน Mutation สั่งให้ Query ที่เกี่ยวข้องโหลดซ้ำอัตโนมัติ

Store setup

เพิ่ม [api.reducerPath]: api.reducer + api.middleware สองบรรทัด แทนที่ Boilerplate ของ Slice ทั้งหมด

ตัวอย่างล่วงหน้า — Session 7: Pattern ขั้นสูง

Session 7 ต่อยอดจาก RTK Query และเพิ่ม Pattern ระดับ Production:

การอัปเดตแบบ Optimistic

อัปเดต UI ทันทีก่อนที่ Server จะตอบกลับ — ย้อนกลับอัตโนมัติหากล้มเหลว

การแบ่งหน้า

ส่งอาร์กิวเมนต์ page และรวมผลลัพธ์ด้วย `serializeQueryArgs` — พร้อมสำหรับ Infinite Scroll

การ Polling

`useGetStudentsQuery(arg, { pollingInterval: 30000 })` — ข้อมูลสดโดยไม่ต้องใช้ WebSockets

Middleware แบบกำหนดเอง

ดักจับ Action สำหรับ Logging, Analytics หรือผลข้างเคียงข้าม Slice

แหล่งอ้างอิงและเอกสารเพิ่มเติม — Session 6

ภาพรวม RTK Query	https://redux-toolkit.js.org/rtk-query/overview	สไลด์ 7-10
เอกสาร <code>createApi</code>	https://redux-toolkit.js.org/rtk-query/api/createApi	สไลด์ 7-9
การโหลดซ้ำอัตโนมัติ (Automated Re-fetching)	https://redux-toolkit.js.org/rtk-query/usage/automated-refetching	สไลด์ 17-18
<code>fetchBaseQuery</code>	https://redux-toolkit.js.org/rtk-query/api/fetchBaseQuery	สไลด์ 8
RTK Query Mutations	https://redux-toolkit.js.org/rtk-query/usage/mutations	สไลด์ 15-16, ขั้นตอน Lab